FraudGuard: Fraud and Phishing Detection for E-Banking (Project Report)

Item	Detail
Author and role	Yashitha Pobbareddy, Security Escalation, GSOC
Purpose	Project report and artifact review pack. It explains what was built, why, how, and what it achieved, and it lists the evidence behind every claim (section 13).
Project type	Personal project: a working prototype plus a proposed production design. The AWS design has not been built.
Version	2.0, November 2026
Repository	https://github.com/Yashitha2001/fraudguard-2
Hosted demo	https://fraudguard-2-esbt.onrender.com

Executive summary

FraudGuard is a working prototype that scores card transactions for fraud and web links for phishing. On simulated data it caught about 80% of fraud, and 88% of the alerts it raised were correct.

The system combines a Random Forest model, a Flask API with a dashboard, and an automated “red team” that attacks the model with simulated fraud to find where it fails. A production design on AWS is proposed in section 10. It is a plan and has not been built.

- **Detection quality:** precision 88.4%, recall 79.6%, F1 0.838 and PR-AUC 0.889 on a held-out, time-ordered test set of 25,703 transactions (392 fraud). A plain model without customer-history features scored F1 0.688.
- **Engineering:** rewriting the feature step to work on whole tables cut its run time by about 98% in local tests. A test proves it gives the same numbers as the one-at-a-time version used by the API.
- **Impact:** the project turned an idea into a tested, documented system with numbers anyone can reproduce, and it found its own weak points before any customer could (section 6).
- **Weak spot:** fraud that imitates the customer is mostly missed. The deployed model caught about 51% of held-out simulated attacks (section 7).
- **Main caveat:** all data is simulated. The results show that the method works. They do not predict performance on real bank traffic.

1. Why I chose this project

I handle security escalations and regularly see the aftermath of fraudulent card transactions and security incidents. Working from that vantage point made me realize I didn't just want to respond to security

failures after they occurred. I wanted to understand how to design, build, test, and scale the automated systems that prevent them in real time.

To bridge that gap and demonstrate full-stack software development capability, I built **FraudGuard**, a real time fraud and phishing detection engine.

I chose fraud and phishing detection for three reasons:

1. **It is a real problem with a clear goal.** Stop stolen card payments and fake bank links without blocking genuine customers.
2. **It uses many engineering skills in one place.** Data, features, a model, an API, tests, security and cloud design all appear in the same project.
3. **It can be measured honestly.** Fraud is rare, so the project forces careful choices about metrics and testing, which is the kind of judgement I wanted to practise.

I added the red team because a model that is only tested on typical cases gives false comfort. I wanted evidence of where it fails, and a way to stop those failures from returning.

What I wanted to learn: how to build features without leaking future information, how to keep training and live code identical, how to design and secure an API, how to test a model and not only code, and how to design a system for scale.

2. Problem and objectives

Online banking fraud is rare but costly. A stolen card is often drained within minutes, so detection has to be fast and must avoid flagging genuine customers.

This project asks one question: can a system decide, at the moment of a transaction, whether it looks like fraud for that particular customer? The objectives were:

1. Score each transaction in real time and give a plain-English reason for every flag.
2. Detects phishing links that trick customers into giving away their login details.
3. Measure quality honestly, using metrics that suit a problem where only about 1.5% of cases are fraud.
4. Test the system against deliberate evasion, not only typical fraud.
5. Offer everything through a secure API and a dashboard that a security analyst can use.

3. How the work was done

The work followed ten steps. Each one produced something that can be checked.

#	Step	What was done and the main choice	Where to check
1	Define success	Chose precision, recall, F1 and PR-AUC instead of accuracy, and a time-based train and test split	`train.py`

#	Step	What was done and the main choice	Where to check
2	Create data	Simulated transactions, because real fraud data is confidential or anonymised. Fraud arrives in short bursts, and 30% is deliberately subtle	`data_gen.py`
3	Build features	Compare each payment with that customer's own history. Written twice: one version for the API, one fast version for training	`features.py`
4	Train and tune	Trained a baseline and a tuned Random Forest. Chose the alert threshold without using the test set	`train.py`, `models/metrics.json`
5	Make it fast	Rewrote feature preparation to work on whole tables and proved the output is identical	`benchmark.py`
6	Serve it	Built a Flask API with API-key checks, input validation, reason codes and a dashboard	`app.py`, `templates/`
7	Test it	28 automated tests, including one that guards the fast and one-at-a-time features against drifting apart	`tests/`
8	Attack it	Built a red team that evolves attacks against the model, finds blind spots and tests fixes	`redteam/`, `red_team.py`
9	Design for production	Drew an AWS architecture and listed what would have to change	Figure 1, section 10
10	Document	Wrote this report, a README and comments in the code	This report, `README.md`

Tools used

Python, pandas and NumPy for data, scikit-learn for the model, Flask for the API, pytest for tests, joblib for saving models, Git and GitHub for version control, and Render for a free demo host.

Development approach and tools. I have taken help from Git to store my code and published it on render, VScode to write code, IEEE publication to change the problem statement and took AI help to pass the test cases and error debugging.

Key decisions and trade-offs

Decision	Alternative	Why this choice
Simulated data	A public dataset	Lets the project run offline and shows customer behaviour clearly. The cost is that results do not transfer to real banks
Split by time	Random split	A model predicts the future in real life, and a random split lets it peek

Decision	Alternative	Why this choice
Random Forest	Gradient boosting or deep learning	Suits table data, needs little preparation and is easy to explain. It was not compared with the others
Two feature builders	One shared builder	One-at-a-time is needed for live requests and the whole table is needed for speed. A test keeps them equal
Customer history in memory	A shared store such as Redis	The simplest thing that works for a prototype. It is a known limit (section 11)
Red team before more tuning	Keep tuning the model	Finding weaknesses first shows where tuning is worth the effort

4. The prototype (implemented system)

The prototype is a single Python service. It loads two trained models at start up and keeps each customer's recent history in memory.

Component	File	What it does
Data generator	`data_gen.py`	Created about 103,000 simulated transactions for 2,500 customers, with fraud (1.5%) arriving in short bursts
Feature builder	`features.py`	Turns a transaction into 10 signals, such as spend versus the customer's average, transactions in the last hour and country mismatch. A fast table-wide version is used for training and a one-at-a-time version for the API
Training	`train.py`	Trains a baseline and a tuned Random Forest, splitting by time (first 75% to train, last 25% to test)
Phishing model	`url_features.py`	Reads 16 signals from the link text, such as length, raw IP address, risky endings and suspicious words, and classifies the link
API and dashboard	`app.py`	Flask service with API-key authentication, batch scoring, reason codes and an alert feed
Integrations	`integrations.py`	Optional Google Safe Browsing lookup and alert webhook, both of which fail safely if unavailable
Red team	`red_team.py`, `redteam/`	Simulated attackers that look for blind spots and test proposed fixes
Tests	`tests/`	28 automated tests (section 8)

How one transaction is scored

1. The API checks the API key and validates the request.
2. It looks up the customer's history and works out the 10 features.

3. The Random Forest returns a risk score between 0 and 1.
4. The score becomes LOW, MEDIUM or HIGH, using the threshold chosen during training (about 0.42).
5. HIGH results get plain-English reasons and go to the alert feed and the optional webhook.

Deployment today: the prototype runs locally with one command. A demo is also hosted on a free Render instance and deployed from GitHub https://fraudguard-2-esbt.onrender.com. The free tier sleeps when idle, so the first request after a pause can take up to about a minute.

5. Worked example: one customer, end to end

This example follows one simulated customer through the system, using real output from the prototype. The customer has 60 past purchases, spends about $39 on average, lives in the US, and usually buys groceries, meals and fuel in person.

Case	What happened	What the system calculated
1. Normal purchase	$31.10 at a grocery store, in person, in the US, on a known device, in the afternoon	The amount is 0.8 times their usual spend, 1 payment in the last hour. Score 0.00, LOW. No alert
2. Stolen-card burst	At 03:10 UTC, four purchases 90 seconds apart from Nigeria on a new device: $520 and $560 in gift cards, $600 in crypto, $640 in electronics	First purchase: 13.4 times usual spend. Score 0.99, HIGH. Five reasons given. By the fourth purchase: 4 payments in an hour. Score 1.00
3. Phishing link	A message contains `http://secure-chase-login.xyz/verify?id=8812`. Another contains the real PayPal help page	Fake link: risky ending, bank words and a long path. Score 1.00, HIGH. Real link: score 0.00, LOW
4. Red-team attack, loud	A simulated smash-and-grab from the UK: four purchases worth $1,206 in electronics and gift cards	First purchase scored 0.96, so it was caught. $0 stolen, because the account freezes at the first flag
5. Red-team attack, disguised	A simulated “full mimic”: three fuel purchases worth $131 in the US, at the customer’s usual hour	Every purchase scored 0.00. Nothing flagged, all $131 stolen. It looks identical to the real customer

The reasons the system gave for the last purchase in case 2 were: amount is 10.1 times the customer’s usual spend; 4 transactions within the last hour; transaction country (NG) differs from the account’s home country; unrecognised device; unusual hour (00:00 to 06:00 UTC); and high-risk merchant category: electronics.

What the example shows. The system works well when fraud looks different from the customer (cases 2 and 4). It cannot tell when fraud looks the same (case 5), and that is the main blind spot found in section 7. The figures come from one run and can vary slightly on another machine.

6. Results and impact

6.1 Model quality

The tuned model improved on the baseline on every measure: recall rose from 61.7% to 79.6% and precision from 77.6% to 88.4%. The figures below come from the held-out test period of 25,703 transactions.

Metric	Baseline	Tuned model	What it means
Precision	77.6%	88.4%	Of the transactions flagged as fraud, this share really were fraud
Recall	61.7%	79.6%	Of all real fraud, this share was caught
F1	0.688	0.838	One score that balances precision and recall
PR-AUC	0.730	0.889	How well fraud is ranked above normal activity across every possible cut-off (1.0 is perfect)
ROC-AUC	0.938	0.981	A looser ranking measure that looks flattering when fraud is rare

In plain numbers, the test period held 392 fraudulent transactions. The model caught about 312 and missed about 80. It wrongly flagged about 41 normal transactions out of roughly 25,300, a false-alarm rate of about 0.16%. These counts are worked out from the rates above, so treat them as approximate.

Accuracy is deliberately not reported. With about 1.5% fraud, a model that flagged nothing would be 98.5% “accurate” and useless.

What drives the decisions. The five most important features were new devices (about 25%), time since the customer’s last transaction (22%), amount (16%), amount compared with the customer’s usual spend (9%) and merchant category (7%). Importance shows what the model relies on, not what causes fraud.

Speed. Rewriting feature preparation to work on whole tables cut its run time by about 98% (roughly 3 to 4 seconds down to 0.05 seconds for 103,000 transactions) in local tests, with identical output. A single API request took about 15 to 20 milliseconds locally, including feature lookup and scoring. These are single machine figures, not production capacity.

Phishing model. It scored 100% on its own synthetic links. That only shows the code path works, because the test links follow the same patterns the model learned. It says nothing about real phishing.

6.2 Impact

This is a personal project, so there are no business users or revenue to report. The impact is what the work measurably achieved and what it demonstrates.

Area	Outcome	Evidence
Detection	About 70 more real frauds were caught in the test period than the baseline (312 against 242), with a higher share of correct alerts. Recall rose 29% and F1 rose 22%, in relative terms	`models/metrics.json`
Speed	Feature preparation about 98% faster, with identical output	`benchmark.py` output
Correctness	A test stops the fast and one-at-a-time features from drifting apart. 28 automated tests in total	`tests/`
Risk discovery	Found that disguised attacks mostly get through (about 51% caught overall) before any customer was exposed, and saved those attacks as permanent tests	`reports/redteam_report.md`
Decision quality	Rejected two of four proposed fixes because they looked good on one measure but caused hidden harm (more false alarms, or worse recall on real fraud)	Section 7, candidate fixes
Skills shown	Data, features, modelling, API design, testing, security thinking and cloud design in one project that can be run and reviewed	Sections 3 to 10

What the impact does not show. It does not show real-world fraud savings, production traffic or results on real bank data. **Why it still matters.** The method is reusable. The same pipeline (event in, history lookup, score, severity, alert) can monitor other event types, such as login activity or access to a customer database (section 10).

7. Red-team (adversarial) evaluation

Against simulated attackers that imitate real customers, the deployed model caught only about 51% of held-out attacks and let about 54% of the attempted money through, although it caught obvious fraud every time.

The red team ran 12 scenarios against 60 customers each. Five were classic patterns, such as a fast smash-and-grab and card testing with tiny charges. Three were deliberate disguises: low-and-slow, daytime camouflage and a full mimic of the customer's habits. Four more were evolved by a search that mutated strategies over six generations, seeing only the model's scores, and kept those that stole the most without being flagged. Over those generations the best attacker's average haul rose from about $700 to about $3,800 per attempt. Every attack ran against a copy of the customer's profile, so no live data was touched, and an account froze at its first flagged transaction.

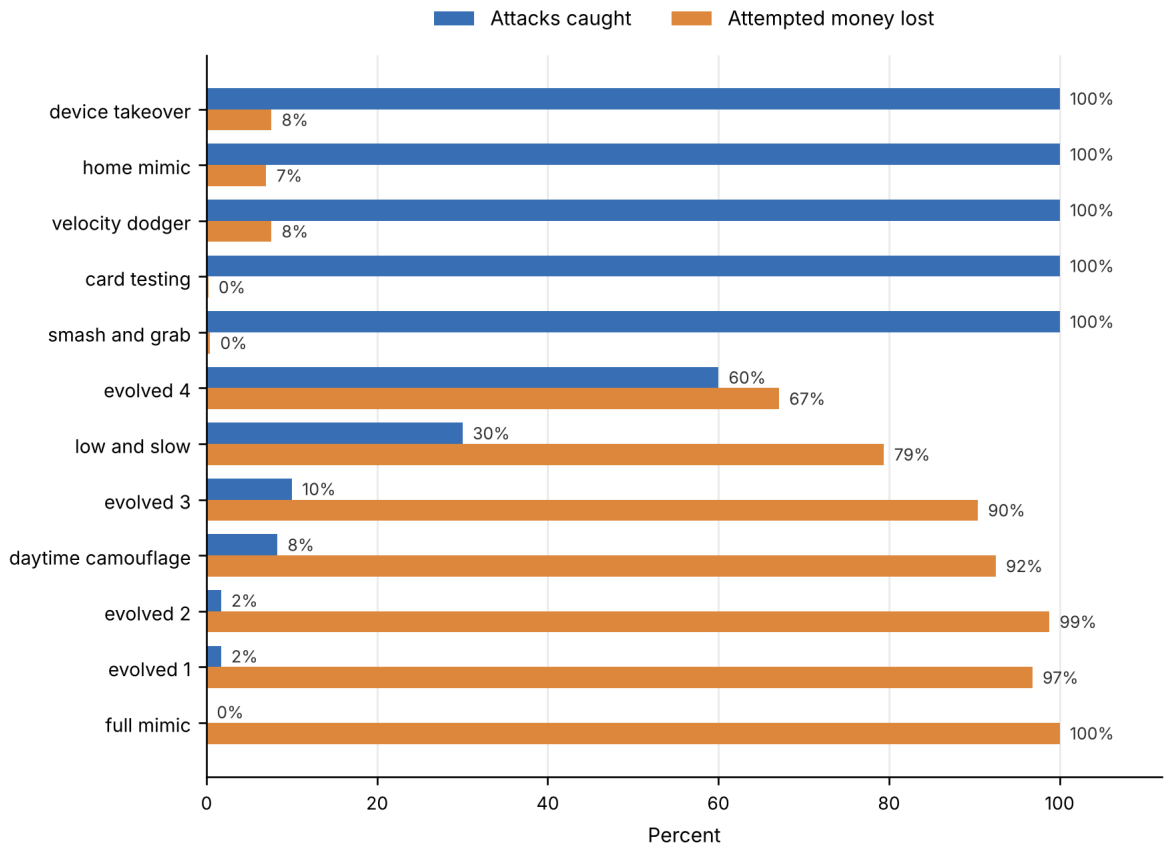


Figure 2. Obvious attacks are caught every time. Disguised and evolved attacks mostly get through.

Blind spots. A small decision tree was fitted to the transactions that slipped through and turned into readable rules:

- **Long gap, known device, web or mobile channel:** about 1,630 transactions missed, and about 73% of normal traffic falls in the same region. This gap is ambiguous: closing it would mean flagging genuine customers.
- **Long gap, new device, amount near the customer's norm:** about 118 transactions missed, and only about 1% of normal traffic matches. This gap is closable at little cost.

Candidate fixes were each retrained and compared on customers and attack types the training never saw, at the same false-alarm rate as the deployed model. Differences carry a 95% bootstrap interval, and a fix is rejected if it raises false positives or loses more than 2 points of real-fraud recall.

Candidate fix	Verdict	Detection change (95% interval)	Reason
Lower the alert threshold	Rejected	+13.2 pts (+11.4 to +15.2)	False alarms rose from 0.17% to 0.65%
Train on red-team attacks	Rejected	+30.7 pts (+28.0 to +33.5)	Recall on real fraud fell from 79.6% to 76.0%, and unseen attack types got 8.5 points worse

Candidate fix	Verdict	Detection change (95% interval)	Reason
Add customer-novelty features	Inconclusive	+0.3 pts (−0.4 to +1.1)	No measurable gain
Novelty features plus attack training	Rejected	+31.2 pts (+28.4 to +34.0)	Recall on real fraud fell from 79.6% to 75.3%

These figures come from one run with a fixed random seed. Running again on another machine can shift them slightly.

No fix was accepted, which is itself a useful result: the quick fixes either buy detection with false alarms or with recall on ordinary fraud. Fraud that copies a customer's habits probably needs new signals, such as device fingerprints or payee history, not more tuning. The attackers here are simulated and limited to the features the model uses, so these rates are not real-world fraud rates.

8. Testing and validation approach

The system is tested at two levels: whether the code does what it should, and whether the model is any good. All 28 automated tests run with one command (`python -m pytest -q tests`). Twenty-one pass and 7 are expected failures that record known blind spots.

Code correctness

- **Feature equivalence.** The fast training version and the one-at-a-time API version of the features must give identical numbers. If they differed, the model would be trained on one thing and scored on another.
- **API behaviour.** A missing key returns 401, bad input returns 400, an empty or oversized batch is refused, an ordinary grocery purchase scores LOW, four fast gift-card purchases from abroad are flagged with reasons, a phishing-style link is flagged and a real PayPal link is not.
- **Red-team logic.** The same random seed gives the same attack, "mimic" attacks copy the customer's habits, card testing starts with tiny charges, a frozen account stops further losses, and mutated strategies stay in realistic ranges.
- **Regression replay.** Attacks found by the red team are saved and replayed. A scenario the model catches must keep being caught, within a 10-point tolerance. A known gap is a strict expected failure, so the suite fails when a change closes it and prompts a refresh.

Model quality

1. **Split by time.** The model trains on the first 75% of the timeline and is tested on the last 25%. This copies real use, where the future is predicted from the past, and prevents leakage.
2. **Metrics suited to rare fraud.** Precision, recall, F1 and PR-AUC are used, not accuracy. Fraud is about 1.5% of rows, so a model that never flags anything would score 98.5% accuracy.
3. **Tuning without peeking.** Settings were chosen inside the training period. The alert threshold comes from out-of-fold predictions, so the test set never influences it.

4. **A baseline.** A plain forest without customer-history features gives the reference point for judging improvement.
5. **Adversarial testing.** The red team (section 7) measures performance against fraud designed to evade the model.

Checked by hand: the dashboard in a browser and the example API client script.

Not yet done: load testing, a security review of the API, and validation on real transaction data.

9. Model choice: why a Random Forest

A Random Forest was chosen because it suits this kind of data and is easy to explain. This section gives the reasoning. It is not a measured comparison against other models.

- **Fraud signals interact.** A large amount matters far more at night, on a new device, in a foreign country. Trees capture these combinations directly.
- **More stable than one tree.** A single tree is easily thrown off by noise. Averaging 150 trees smooths that out.
- **Little preparation.** Skewed amounts and mixed feature types needed no scaling or special handling.
- **Explainable.** Feature importance and simple rules support the plain-English reason shown with each alert.
- **Practical.** It is mature in scikit-learn, easy to save and load, and scores a transaction in milliseconds.

How it was tuned. A random search tried 8 combinations of settings with 3-fold cross-validation inside the training period, scored on PR-AUC. The chosen model uses 150 trees with at least 2 samples per leaf.

What to test next. Gradient boosting is the most likely rival and should be tuned and compared on the same split. An unsupervised model such as Isolation Forest could add a second layer for fraud types nobody has labelled yet. Deep learning was not tried: with about 100,000 rows and 10 numeric features it rarely beats tree models and is harder to explain.

10. Proposed AWS production architecture

The proposed design moves each prototype component onto a managed AWS service so the system can scale, survive failures and be updated safely. This is a design only. None of it is deployed.

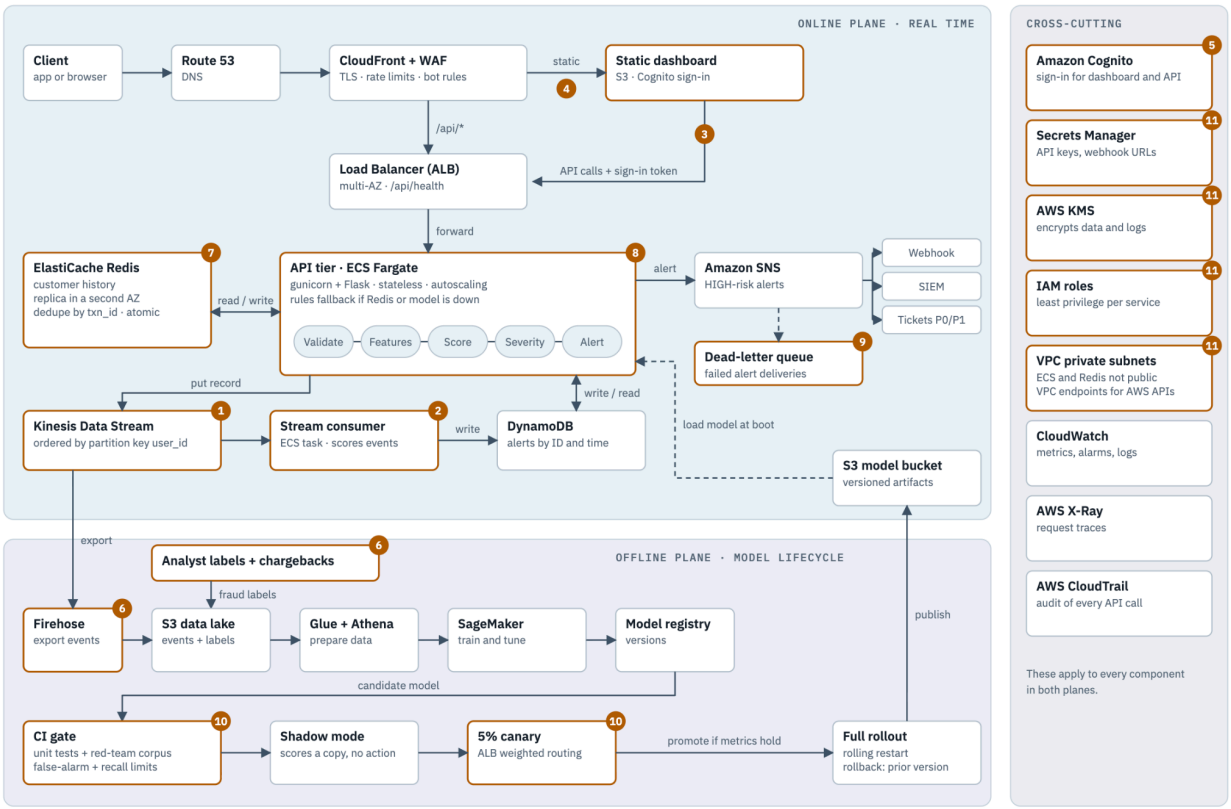


Figure 1. Proposed AWS architecture. The online plane (top) handles live requests, the offline plane (bottom) trains and releases models, and the right-hand column shows controls that apply everywhere.

Reading the figure. The orange numbers mark design details that matter most. They are explained in the key below. In this design the API tier returns its decision first and then publishes the event to Kinesis, so response time does not depend on the stream. The dashboard calls the same API with a sign-in token, using read-only endpoints.

Figure 1 key

#	Component	What it does and why
1	Kinesis Data Stream	Receives one event per transaction, ordered by customer ID, so each customer's history is updated in the right order
2	Stream consumer	An ECS task that scores events from the stream and writes alerts, so heavy traffic never slows the live request path
3	Dashboard calls to the API	The dashboard sends a sign-in token with every call through the load balancer
4	Static dashboard	Served from S3 through CloudFront. It holds no secrets
5	Amazon Cognito	Sign-in for analysts on the dashboard and the API
6	Feedback loop	Analyst labels and chargebacks, plus events exported by Firehose, flow into the S3 data lake so the model can be retrained on real outcomes

#	Component	What it does and why
7	ElastiCache for Redis	Holds customer history. Has a replica in a second zone and ignores repeated transaction IDs, so a duplicate event is not counted twice
8	API tier on ECS Fargate	Stateless and autoscaling. If Redis or the model is unavailable, it falls back to simple rules instead of failing
9	Dead-letter queue	Keeps alerts that could not be delivered, so none are lost
10	Release gates	A CI gate (unit tests, red-team corpus, and limits on false alarms and recall) and a 5% canary with automatic promotion only if metrics hold
11	Security controls	Secrets Manager for keys, KMS for encryption, least-privilege IAM roles, and private subnets with VPC endpoints so traffic stays inside AWS

Request path

1. A client calls over HTTPS. Route 53 sends the request to CloudFront with AWS WAF, which handles TLS, rate limits and bot rules.
2. Requests for `/api/*` go to an Application Load Balancer spread across several Availability Zones. It checks `/api/health` and routes traffic to healthy containers.
3. The API tier runs on ECS Fargate (gunicorn and Flask). Each request is validated, enriched with the customer's history from Redis, scored, given a severity and, if needed, turned into an alert.
4. Alerts go to Amazon SNS, which sends them on to a webhook, a security monitoring system (SIEM) and ticketing. HIGH maps to priority P0 and MEDIUM to P1. Failed deliveries go to the dead-letter queue.

Streaming path. After returning a decision, the API publishes the event to Kinesis with the customer ID as the partition key. A consumer scores events and writes alerts to DynamoDB. Partitioning by customer keeps each customer's transactions in order, which the history features need.

Model path (offline plane). Events are exported to an S3 data lake together with analyst labels and chargebacks, prepared with Glue and Athena, and trained and tuned in SageMaker. The result goes into a model registry, then through the CI gate, shadow mode (it scores a copy and takes no action), a 5% canary and full rollout. Rolling back means pointing to the previous version. The API loads the versioned model from S3 at start-up.

Operations. CloudWatch metrics and alarms, X-Ray traces, structured and redacted logs encrypted with KMS, and CloudTrail for audit.

How the prototype maps to AWS

Prototype today	AWS service	Reason for the change
Flask and gunicorn on Render	ECS Fargate behind an Application Load Balancer	Autoscaling, multi-zone resilience and no servers to manage
Customer history in process memory	ElastiCache for Redis	Shared by all containers and kept across restarts

Prototype today	AWS service	Reason for the change
Last 100 alerts in memory	DynamoDB	Durable alert storage and history
Background webhook thread	Amazon SNS with a dead-letter queue	Reliable fan-out with retries to webhook, SIEM and tickets
Model files in the repository	Versioned S3 bucket and model registry	Version control and one-step rollback
`train.py` run by hand	Glue, Athena and a SageMaker pipeline	Repeatable training on larger data
Red-team tests run with pytest	CI gate before rollout	Blocks a model that gets worse against known attacks
`/api/health` endpoint	Load balancer health check	Unhealthy containers are replaced automatically
Dashboard served by Flask	S3 and CloudFront static site with Cognito sign-in	Cheap, fast, and takes load off the API
Console logs	CloudWatch, X-Ray and CloudTrail	Monitoring, tracing and audit

Where the same design can be used

The pipeline (event in, history lookup, score, severity, alert) fits other systems. Only the feature calculation changes.

Use	How it connects	Output
Card payments	Real time call before a payment is approved	Allow, ask for extra verification, or block
Login and activity logs	Stream consumer reading events as they arrive	Alerts within seconds
Email and web links	Plug-in or gateway extension	Quarantine or rewrite the link
A database or shared code package used by customers	Access events scored against each caller's usual behaviour	P0 and P1 risks on a dashboard

The prototype already produces risk levels and reasons. The P0 and P1 routing and the extra feature calculations for other event types would be new work.

Security and logging

- **Log decisions, not data.** Logs should hold the transaction ID, a protected customer ID, score, level, reason codes, model version and response time. They should never hold card numbers, personal details or full links with query strings. The prototype's alert line currently logs a raw customer ID. Production would replace it with a keyed hash (HMAC) using a rotating KMS-managed key, or a token.

- **Protect the logs themselves.** Production logs are sensitive. They should sit in a separate account and be readable only through audited, multi-factor roles, with time limited emergency access.
- **Keep card data out of scope.** The scoring service should receive tokens and derived features, never card numbers. This reduces the compliance burden (for example PCI DSS).

Production details to settle before building

- **Alert table design.** Key alerts by customer ID and timestamp for per-customer lookups. For a "latest alerts" view, add a second index whose key includes a date bucket, not one fixed value, so writes spread across partitions instead of creating a hot partition. Expire old alerts automatically with a time-to-live (for example 90 days).
- **Sign-in at the edge.** Analysts sign in through Cognito. Verify tokens at the entry point, either with the load balancer's built-in authentication or an API Gateway authorizer. Other services call with signed requests. The choice depends on traffic and cost.
- **Cache misses.** If a customer's history is not in Redis, read it from a durable store and use a short lived lock so only one container rebuilds it. This stops many containers hitting the database at once (a "thundering herd").
- **Model drift.** Use SageMaker Model Monitor to compare live inputs with the training data and raise a CloudWatch alarm that starts a retraining review. Fraud patterns change, so a model decays without this.
- **Private access and secrets.** Fargate tasks use an execution role to pull the image and secrets at start-up and a least-privilege task role at run time. VPC endpoints keep calls to S3, Secrets Manager and Kinesis inside the AWS network.
- **Safe rollouts.** Deploy with rolling updates. The load balancer drains old containers while new ones load the model and pass the health check.
- **One copy of the feature code.** Use the same feature code in the training pipeline and the API, so the model is never trained on numbers computed differently from the ones it sees live.
- **Cost.** Fargate, Redis and Kinesis have fixed monthly costs. Estimate them, and run a load test, before committing.

11. Limitations, risks and responsible use

The biggest limitation is that every number in this report comes from simulated data, so none of it predicts how the system would perform on real bank traffic.

- **Simulated data.** The fraud patterns were written by the same person who built the model's features, which favours the model. Real fraud is messier and changes over time.
- **One test split.** The headline figures come from a single time-ordered split without confidence intervals, so small differences are not proven.
- **The features set the ceiling.** Fraud that mimics the customer's own habits is mostly invisible (section 7).

- **Drift.** Fraudsters adapt, so a model trained once will decay. A real deployment needs monitoring and scheduled retraining.
- **Prototype shortcuts.** Customer history lives in one process's memory and is lost on restart. One lock guards all updates. A repeated event would be counted twice. The default API key is a development placeholder. The free hosting tier sleeps when idle. None of this is acceptable for production.
- **Phishing model.** It was trained and tested on synthetic links, so its 100% score has no real-world meaning.
- **No independent review.** This is a solo project, so there are no formal code reviews.
- **Responsible use.** A fraud score should support a human decision, not replace it. Signals such as country can flag some customers more often than others, so a real deployment would need a fairness review, a way for customers to dispute a block, and handling of card and personal data that follows the applicable rules (for example PCI DSS and privacy law), which are outside the scope of this prototype.

12. Roadmap

The next work should first make the results credible, then make the system production-ready, and only then make it adapt on its own.

1. **Make the results credible.** Retrain on a public real-world dataset such as PaySim, add confidence intervals, tune gradient boosting against the Random Forest, and add load and security testing.
2. **Make it production-ready.** Build the AWS design in section 10 in stages: shared customer history in Redis, durable alerts in DynamoDB, versioned models in S3, and a release pipeline that blocks any model that regresses on the red-team corpus.
3. **Make it adapt.** Monitor score drift, feed analyst decisions back as new labels, release new models through shadow mode and a 5% canary, and add signals aimed at the blind spots found by the red team.

13. Artifacts and evidence

This section lists every artifact a reviewer can open, and how to check each claim in this report.

13.1 Artifact index

Artifact	What it is	How to check it
Source code	All Python and HTML for the prototype	GitHub https://github.com/Yashitha2001/fraudguard-2. Clone it and follow the README
Hosted demo	The dashboard and API on a free Render instance	https://fraudguard-2-esbt.onrender.com. Click "Fire suspicious burst", then scan one fake and one real link

Artifact	What it is	How to check it
Test suite	28 automated tests in `tests/`	`Python -m pytest -q tests` should show 21 passed and 7 expected failures
Training results	`models/metrics.json`	Created by `python train.py`. Compare the baseline and tuned figures
Speed benchmark	Output of `benchmark.py`	`python benchmark.py` prints both timings and whether the outputs match
Red-team report	`reports/redteam_report.md` and `.json`	Created by `python red_team.py` (about 40 seconds)
Architecture diagram	Figure 1 in this report	Section 10, with the key to the numbered details
Design and review history	No formal design doc or code reviews exist, because this is a solo project	This report is the design record. https://github.com/Yashitha2001/fraudguard-2

13.2 Claims and evidence

Claim	Evidence	How to reproduce
Recall 79.6% and precision 88.4% on held-out data	`models/metrics.json`	`python train.py`
Feature preparation about 98% faster, same output	`benchmark.py` output	`python benchmark.py`
Fast and one-at-a-time features match	Test: vectorised features match row by row	`python -m pytest -q tests`
About 51% of held-out attacks caught	`reports/redteam_latest.json`	`python red_team.py`
Two of four candidate fixes rejected	`reports/redteam_report.md`, section 5	`python red_team.py`
Fake link flagged, real link not	Test: URL scan, and the worked example	`python -m pytest -q tests`

Results can differ slightly between machines and library versions. Quote the numbers your own run prints.

Glossary

Term	Plain meaning
API	A front desk that lets other programs send a request and get an answer
Feature	One fact calculated about a transaction and used as a clue by the model
Random Forest	A model made of many simple decision trees that vote

Term	Plain meaning
Precision	Of the payments flagged, the share that were truly fraud
Recall	Of all fraud, the share that was caught
F1 score	One number that is high only when precision and recall are both high
PR-AUC	How well the model ranks fraud above normal payments
Threshold	The score at or above which a payment is flagged
False positive	A genuine payment wrongly flagged
Red team	A group, or here a program, that attacks a system to find its weak points
Regression test	A test that stops an old problem from returning
Stateless	A server that keeps no customer data inside itself
Shadow mode / canary	Trying a new model silently, then on 5% of traffic, before a full release
P0 / P1	Priority levels, where P0 is the most urgent